
Java + Databases

— JDBC. JPA. Hibernate. —
Spring Data

JDBC

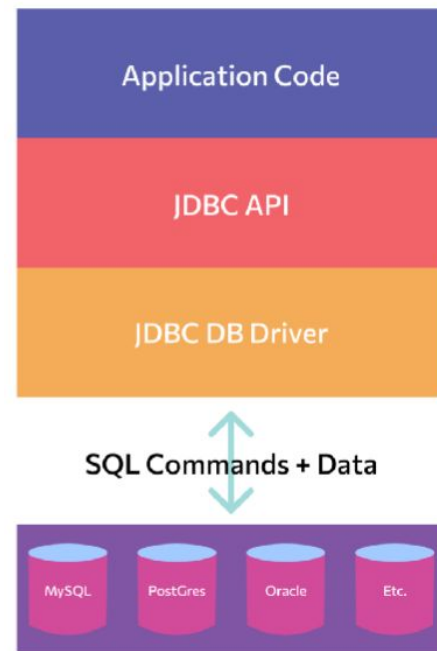
JDBC — это платформенно независимый промышленный стандарт взаимодействия Java-приложений с реляционными базами данных. Впервые был включен в состав JDK 1.1 в 1997 году. JDBC управляет:

- подключением к базе данных;
- выдачей запросов и команд;
- обработкой данных, полученных из базы

Пакет JDBC состоит из двух главных компонентов:

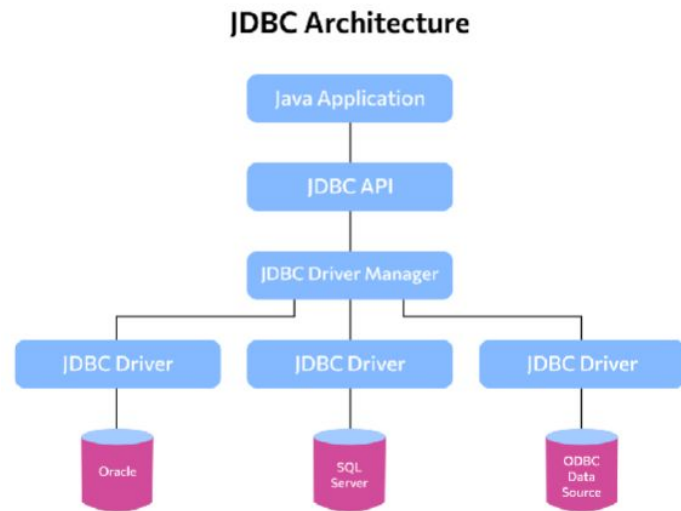
1. API (программного интерфейса), который поддерживает связь между Java-приложением и менеджером JDBC;
2. Драйвера JDBC, который поддерживает связь между менеджером JDBC и драйвером базы данных.

Соединение с базой устанавливается по особому URL. При этом разработчику не нужно знать специфику конкретной базы — API выступает в качестве посредника между базой и приложением. Это упрощает как процесс создания приложения, так и переход на базу данных другого типа.



Этапы подключения к БД

1. Установка базы данных на сервер или выбор облачного сервиса, к которому нужно получить доступ.
2. Подключение библиотеки JDBC.
3. Проверка факта нахождения необходимого драйвера JDBC в classpath.
4. Установление соединения с базой данных с помощью библиотеки JDBC.
5. Использование установленного соединения для выполнения команд SQL.
6. Закрытие соединения после окончания сеанса.



Элементы JDBC

- Менеджер драйверов (Driver Manager)

Этот элемент управляет списком драйверов БД. Каждой запрос на соединение требует соответствующего драйвера. Первое совпадение даёт нам соединение.

- Драйвер (Driver)

Этот элемент отвечает за связь с БД. Работать с ним нам приходится крайне редко. Вместо этого мы чаще используем объекты DriverManager, которые управляют объектами этого типа.

- Соединение (Connection)

Этот интерфейс обеспечивает нас методами для работы с БД. Все взаимодействия с БД происходят исключительно через Connection.

- Выражение (Statement)

Для подтверждения SQL-запросов мы используем объекты, созданные с использованием этого интерфейса.

- Результат (ResultSet)

Экземпляры этого элемента содержат данные, которые были получены в результате выполнения SQL – запроса. Он работает как итератор и “пробегают” по полученным данным.

- Исключения (SQL Exception)

Этот класс обрабатывает все ошибки, которые могут возникнуть при работе с БД.

Statements

Statements используется для выполнения обычных SQL-запросов. Вы не можете передавать параметры в SQL-запрос во время выполнения с помощью этого интерфейса. Этот интерфейс предпочтительнее двух других интерфейсов, если вы выполняете конкретный запрос SQL только один раз. Производительность этого интерфейса также намного ниже по сравнению с двумя другими интерфейсами. В большинстве случаев интерфейс Statement используется для операторов DDL, таких как CREATE , ALTER , DROP и т.д.

PreparedStatement

PreparedStatement используется для выполнения динамических или параметризованных SQL-запросов. PreparedStatement расширяет интерфейс Statement. Можно передать параметры в SQL-запрос во время выполнения, используя этот интерфейс. Рекомендуется использовать PreparedStatement, если определенный SQL-запрос выполняется несколько раз. Он обеспечивает лучшую производительность, чем интерфейс Statement. Потому что PreparedStatement предварительно скомпилирован, а план запроса создается только один раз, независимо от того, сколько раз выполняется этот запрос.

CallableStatement

CallableStatement используется для выполнения хранимых процедур. CallableStatement расширяет PreparedStatement. Используя CallableStatement, можно передавать хранимым процедурам 3 типа параметров: IN - используется для передачи значений хранимой процедуре, OUT - используется для хранения результата, возвращаемого хранимой процедурой, а IN OUT - действует как параметр IN и OUT. Перед вызовом хранимой процедуры необходимо зарегистрировать параметры OUT с помощью метода registerOutParameter() CallableStatement. Производительность этого интерфейса выше, чем у двух других интерфейсов. Потому что он вызывает хранимые процедуры, которые уже скомпилированы и хранятся на сервере базы данных.

Statements

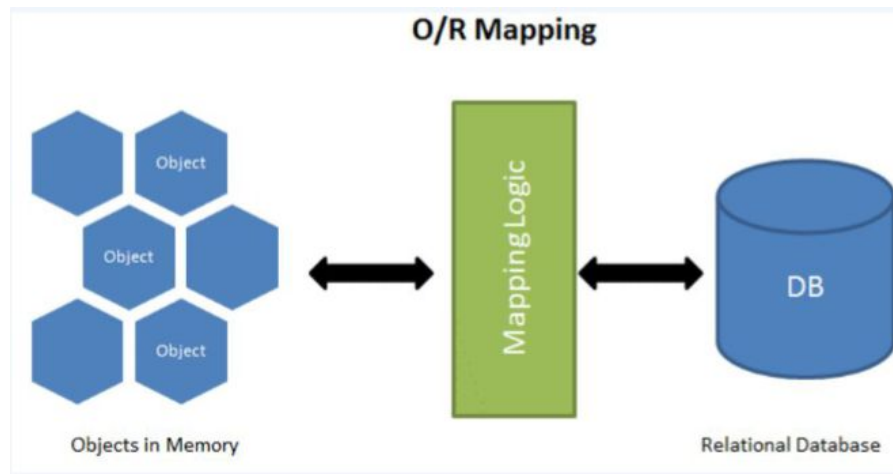
Statement	PreparedStatement	CallableStatement
Он используется для выполнения обычных SQL-запросов.	Он используется для выполнения параметризованных или динамических SQL-запросов.	Он используется для вызова хранимых процедур.
Предпочтительно, когда конкретный SQL-запрос должен выполняться только один раз.	Предпочтительно, когда конкретный запрос должен выполняться несколько раз.	Предпочтительно, когда должны выполняться хранимые процедуры.
Вы не можете передавать параметры в SQL-запрос с помощью этого интерфейса.	Вы можете передать параметры в SQL-запрос во время выполнения, используя этот интерфейс.	С помощью этого интерфейса вы можете передавать 3 типа параметров. Это - IN, OUT и IN OUT.
Этот интерфейс в основном используется для операторов DDL, таких как CREATE, ALTER, DROP и т. Д.	Он используется для любого типа SQL-запросов, которые должны выполняться несколько раз.	Он используется для выполнения хранимых процедур и функций.
Производительность этого интерфейса очень низкая.	Производительность этого интерфейса лучше, чем у интерфейса Statement (когда он используется для многократного выполнения одного и того же запроса).	Производительность этого интерфейса высока.

JDBC.Пример

```
6  ✓ public class JDBC {
7  ✓      public static void main(String[] args) {
8          String url="jdbc:mysql://localhost:3306/tutorial";
9          String username="root";
10         String password="220620011111";
11         try {
12             Class.forName("com.mysql.jdbc.Driver");
13             Connection connection= DriverManager.getConnection(url, username, password);
14             Statement statement=connection.createStatement();
15             ResultSet resultSet=statement.executeQuery("SELECT * FROM employee");
16             while (resultSet.next()){
17                 System.out.println(resultSet.getString(2));
18             }
19             resultSet.close();
20             statement.close();
21             connection.close();
22         } catch (SQLException e) {
23             e.printStackTrace();
24         } catch (ClassNotFoundException e) {
25             e.printStackTrace();
26         }
27     }
28 }
```

ORM

ORM (Object-Relational Mapping) - технология, которая позволяет разработчикам работать с объектами в объектно-ориентированном стиле, в то время как данные хранятся в реляционных базах данных. ORM устанавливает соответствие (отображение) между объектами и таблицами базы данных, что позволяет автоматически сохранять, извлекать, обновлять и удалять данные без явного написания SQL-запросов.



JPA

JPA (Java Persistence API) - это спецификация Java EE, которая определяет стандартный способ работы с объектами в Java-приложениях и их сохранения в реляционных базах данных. JPA предоставляет набор аннотаций и интерфейсов для работы с объектно-реляционным отображением (ORM).

Основная цель JPA состоит в том, чтобы абстрагировать приложение от деталей конкретной базы данных и предоставить унифицированный интерфейс для работы с данными. Он позволяет разработчикам работать с объектами в объектно-ориентированном стиле, в то время как JPA-провайдеры (например, Hibernate, EclipseLink) обеспечивают преобразование этих объектов в соответствующие записи в базе данных и обратно.

JPA не является самостоятельной реализацией, а является набором спецификаций. Для использования JPA необходимо подключить JPA-провайдер, такой как Hibernate, EclipseLink или другие, который реализует спецификацию JPA и предоставляет конкретную реализацию ORM-функциональности.

<https://docs.oracle.com/javaee/7/api/javax/persistence/package-summary.html>

JPA основные интерфейсы

1. **EntityManager** - интерфейс является основным интерфейсом для выполнения операций по сохранению, чтению, обновлению и удалению сущностей в базе данных. Он предоставляет методы, такие как `persist()`, `find()`, `merge()`, `remove()`, для выполнения этих операций, а также методы для управления транзакциями.
2. **EntityTransaction** - интерфейс представляет транзакцию базы данных и предоставляет методы для управления транзакционными операциями, такими как `begin()`, `commit()`, `rollback()`. Он используется для управления границами транзакций и обеспечения согласованности изменений в базе данных.
3. **Query** - интерфейс позволяет создавать и выполнять запросы к базе данных. Он предоставляет методы для создания запросов на основе языка запросов, таких как JPQL (Java Persistence Query Language) или Criteria API. Методы `getResultList()` и `getSingleResult()` используются для получения результатов запроса.
4. **TypedQuery** - интерфейс является расширением интерфейса `Query` и предоставляет типизированные методы для работы с запросами. Он позволяет указать ожидаемый тип результата запроса и предоставляет более типобезопасные операции.
5. **CriteriaQuery** - интерфейс представляет типобезопасный способ создания запросов с использованием Criteria API. Он позволяет строить запросы в программном коде с помощью объектов-критериев, представляющих различные части запроса, такие как выборка, условия и сортировка.
6. **CriteriaBuilder** - интерфейс предоставляет методы для создания объектов-критериев (`CriteriaQuery`, `Predicate`, `Order` и других) и выполнения операций сравнения, логических операций и математических операций в запросах.

JPA основные интерфейсы

JPA Entity

JPA Entity (сущность) - это Java-класс, который отображается на соответствующую таблицу в базе данных.

JPA использует аннотации для определения сущности и ее отображения на таблицу.

Может в себе содержать:

Value types:

- базовые типы
- вложенные типы
- коллекции

Entity types

Требования Entity

- 1) Entity класс должен быть отмечен аннотацией Entity или описан в XML файле конфигурации JPA,
- 2) Entity класс должен содержать public или protected конструктор без аргументов (также может иметь конструкторы с аргументами).
- 3) Entity класс должен быть классом верхнего уровня (top-level class).
- 4) Entity класс не может быть enum или интерфейсом.
- 5) Entity класс не может быть финальным классом (final class).
- 6) Entity класс не может содержать финальные поля или методы, если они участвуют в маппинге (persistent final methods or persistent final instance variables).
- 7) Если объект Entity класса будет передаваться по значению как отдельный объект (detached object), например через удаленный интерфейс (through a remote interface), он так же должен реализовывать Serializable интерфейс.
- 8) Поля Entity класс должны быть напрямую доступны только методам самого Entity класса и не должны быть напрямую доступны другим классам, использующим этот Entity. Такие классы должны обращаться только к методам (getter/setter методам или другим методам бизнес-логики в Entity классе).
- 9) Entity класс должен содержать первичный ключ, то есть атрибут или группу атрибутов которые уникально определяют запись этого Entity класса в базе данных

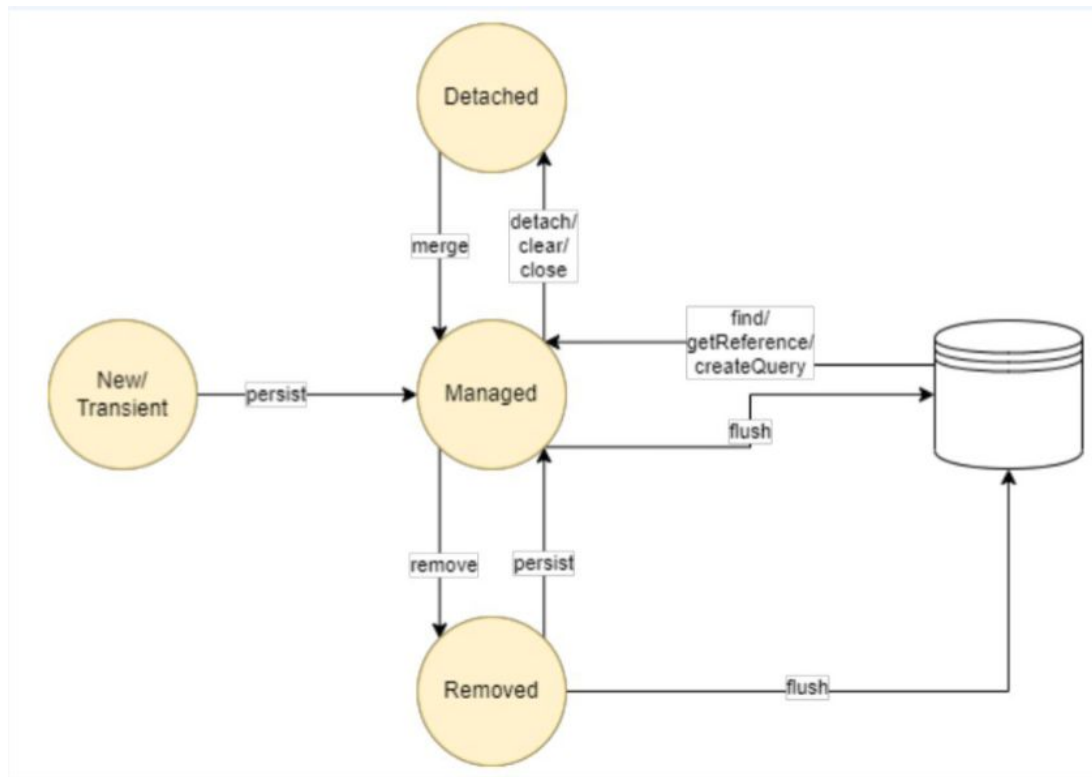
Entity основные аннотации

1. @Entity - применяется к классу и указывает, что данный класс является сущностью. Он должен иметь конструктор без аргументов и быть открытым для наследования или наследоваться от другого класса.
2. @Table - применяется к классу и используется для указания имени таблицы, к которой будет отображаться сущность. Если имя таблицы не указано, то оно будет соответствовать имени класса.
3. @Id - применяется к полю или геттеру/сеттеру и указывает, что это поле является первичным ключом сущности.
4. @GeneratedValue - используется в сочетании с @Id и указывает, что значение первичного ключа будет генерироваться автоматически.
5. @Column - применяется к полю или геттеру/сеттеру и используется для указания отображения поля на столбец таблицы. Она также позволяет указать различные атрибуты, такие как имя столбца, тип данных, ограничения и др.
6. @OneToMany и @ManyToOne - используются для определения отношений "один-ко-многим" и "многие-к-одному".
7. @ManyToMany - применяется для определения отношения "многие-ко-многим" между сущностями.
8. @JoinColumn - используется в сочетании с @ManyToOne или @OneToOne и указывает столбец, который используется для связи между сущностями.

JPA состояния

1. **Transient** - сущность находится в состоянии transient, когда она только что создана или не связана с текущим персистентным контекстом. В этом состоянии сущность не отслеживается JPA и не имеет соответствующей записи в базе данных.
2. **Persistent** - сущность находится в состоянии persistent, когда она ассоциирована с персистентным контекстом JPA. В этом состоянии изменения, внесенные в сущность, отслеживаются JPA, и они могут быть автоматически синхронизированы с базой данных во время фиксации транзакции.
3. **Detached** - сущность находится в состоянии detached, когда она была отсоединена от персистентного контекста JPA. Это может произойти, например, когда транзакция, в рамках которой была получена сущность, завершилась, или явно вызван метод отсоединения. В этом состоянии изменения в сущности не отслеживаются JPA, и они не автоматически синхронизируются с базой данных.
4. **Removed** - сущность находится в состоянии removed, когда она была помечена для удаления. В этом состоянии JPA удалит соответствующую запись из базы данных при фиксации транзакции.

JPA состояния entity





Hibernate

Hibernate - это одна из самых популярных ORM (Object-Relational Mapping) библиотек для языка программирования Java. Он предоставляет удобные и мощные инструменты для работы с базами данных, позволяя разработчикам взаимодействовать с данными в объектно-ориентированном стиле, не вдаваясь в детали работы с SQL.

HQL (Hibernate Query Language) Query

- объектно-ориентированный язык запросов, подобный SQL, но работающий с объектами сущностей. HQL позволяет выполнять запросы, которые оперируют объектами и их свойствами, а не непосредственно таблицами базы данных.

```
String hql = "FROM User WHERE age > :age";
```

```
Query query = session.createQuery(hql);
```

```
query.setParameter("age", 18);
```

```
List<User> users = query.list();
```

Criteria Query

Criteria API позволяет создавать запросы с использованием критериев и условий, а не явно писать SQL или HQL запросы. Он предоставляет типобезопасные и более гибкие возможности для создания запросов.

```
CriteriaBuilder builder = session.getCriteriaBuilder();  
CriteriaQuery<User> criteriaQuery =  
builder.createQuery(User.class);  
Root<User> root = criteriaQuery.from(User.class);  
criteriaQuery.select(root).where(builder.greaterThan(root.get("age"), 18));  
List<User> users = session.createQuery(criteriaQuery).getResultList();
```

Native SQL Query

Hibernate также позволяет выполнять нативные SQL запросы, которые напрямую работают с таблицами базы данных.

```
String sql = "SELECT * FROM users WHERE age > :age";
```

```
Query query = session.createNativeQuery(sql, User.class);
```

```
query.setParameter("age", 18);
```

```
List<User> users = query.getResultList();
```

Named Query

Можно определить именованные запросы в классе сущности или в файле XML конфигурации Hibernate. Затем вы можете вызывать эти именованные запросы, используя их уникальное имя.

```
@NamedQueries({
    @NamedQuery(name = "User.findByAge", query = "FROM User WHERE age > :age")
})
public class User {
    // ...
}

Query query = session.getNamedQuery("User.findByAge");
query.setParameter("age", 18);
List<User> users = query.list();
```

Типы связей

- OneToOne - один к одному
- OneToMany - один ко многим
- ManyToOne - многие к одному
- ManyToMany - многие ко многим

- Односторонние
- Двусторонние

One-to-One

Односторонняя

```
@Entity
public class User {
    @OneToOne
    @JoinColumn(name = "address_id")
    private Address address;
    // ...
}

@Entity
public class Address {
    // ...
}
```

Двунаправленная

```
@Entity
public class User {
    @OneToOne(mappedBy = "user")
    private UserProfile userProfile;
    // ...
}

@Entity
public class UserProfile {
    @OneToOne
    @JoinColumn(name = "user_id")
    private User user;
    // ...
}
```


One-to-Many

Односторонняя

```
@Entity
public class User {
    @OneToMany
    @JoinColumn(name = "user_id")
    private List<Order> orders;
    // ...
}
```

Двунаправленная

```
@Entity
public class User {
    @OneToMany(mappedBy = "user")
    private List<Order> orders;
    // ...
}

@Entity
public class Order {
    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;
    // ...
}
```

Many-to-Many

```
@Entity
public class User {
    @ManyToMany
    @JoinTable(name = "user_role",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"))
    private List<Role> roles;
    // ...
}

@Entity
public class Role {
    @ManyToMany(mappedBy = "roles")
    private List<User> users;
    // ...
}
```

Нibernate наследование

- 1) Одна таблица для каждого класса
- 2) Одна таблица для каждого класса через Union
- 3) Единая таблица для всей иерархии классов
- 4) Одна таблица для каждого класса с использованием соединений (JOIN)

Hibernate cache

1. Кэш первого уровня (First-level Cache) - кэш первого уровня в Hibernate является встроенным кэшем, который автоматически активируется и управляется Hibernate для каждой текущей сессии (Session). Он хранит объекты сущностей, полученные в рамках текущей сессии, и предотвращает повторные запросы к базе данных при обращении к одному и тому же объекту в течение сессии.
2. Кэш второго уровня (Second-level Cache) - кэш второго уровня в Hibernate является распределенным кэшем, который доступен для всех сессий и сохраняется между ними. Он может быть настроен для хранения повторно используемых данных сущностей или запросов, чтобы избежать повторных обращений к базе данных. Для работы с кэшем второго уровня необходимо использовать сторонние поставщики кэша, такие как Ehcache, Infinispan, Hazelcast и другие.
3. Кэш запросов (Query Cache) - кэш запросов в Hibernate позволяет кэшировать результаты запросов. Это особенно полезно для запросов, которые выполняются часто и возвращают стабильные результаты. Кэш запросов может быть включен и настроен для конкретных запросов с использованием соответствующей настройки.

Проблема N+1

Проблема N+1 является распространенной проблемой в Hibernate (и других ORM-фреймворках), которая возникает при загрузке связанных данных из базы данных. Эта проблема может привести к неэффективной работе и снижению производительности приложения.

Проблема N+1 возникает в следующих случаях:

1. Загрузка связанных коллекций: когда вы загружаете сущность, которая имеет связанную коллекцию (например, One-to-Many или Many-to-Many отношение), и для каждого элемента в коллекции выполняется отдельный запрос к базе данных. Это может привести к большому количеству запросов, что замедляет работу приложения.
2. Ленивая загрузка: если вы используете ленивую загрузку (lazy loading) для связанных данных, то при обращении к этим данным Hibernate выполняет дополнительные запросы для загрузки связанных объектов, что может привести к N+1 проблеме.

Проблема N+1

- Join fetch
- Entity graph
- @FetchMode (SUBSELECT, JOIN)
- @BatchSize

Примеры:

Batch https://www.youtube.com/watch?v=nuTI3O06kUc&ab_channel=MissXing

Entity Graph https://www.youtube.com/watch?v=qPE98hZwBXA&t=1s&ab_channel=MissXing

Join Fetch https://www.youtube.com/watch?v=J2BC9zEA4U4&t=8s&ab_channel=MissXing

FetchMode https://www.youtube.com/watch?v=V0tjqtgLIBw&ab_channel=MissXing

Spring Data - что это

Spring Data — дополнительный удобный механизм для взаимодействия с сущностями базы данных, организации их в репозитории, извлечение данных, изменение, в каких то случаях для этого будет достаточно объявить интерфейс и метод в нем, без имплементации.

Преимущества Spring Data JPA:

- имеет в себе реализацию CRUD операций

- имеет реализацию для большинства популярных баз данных

- имеет реализацию query builder mechanism

- возможность создавать named queries

Spring Repository

Основное понятие в Spring Data — это репозиторий. Это несколько интерфейсов которые используют JPA Entity для взаимодействия с ней.

Так например интерфейс

public interface CrudRepository<T, ID extends Serializable> extends Repository<T, ID>
обеспечивает основные операции по поиску, сохранения, удалению данных (CRUD операции):

- *T save(T entity);*
- *Optional findById(ID primaryKey);*
- *void delete(T entity);*

и др. операции.

Есть и другие абстракции, например *PagingAndSortingRepository*.

1 Repository<T, ID>

1 CrudRepository<T, ID>

saveAll (Iterable <S>) Iterable <S>
existsById (ID) boolean
findAll () Iterable <T>
save(S) S
count() long
deleteAllById (Iterable <ID>) void
deleteById (ID) void
findById (ID) Optional <T>
delete (T) void
deleteAll (Iterable <T>) void
deleteAll () void
findAllById (Iterable <ID>) Iterable <T>

1 PagingAndSortingRepository<T, ID>

findAll (Sort) Iterable <T>
findAll (Pageable) Page <T>

1 JpaRepository<T, ID>

getById (ID) T
findAll () List <T>
findOne (ID) T
saveAll (Iterable <S>) List <S>
saveAndFlush (S) S
deleteAllByIdInBatch (Iterable <ID>) void
saveAllAndFlush (Iterable <S>) List <S>
findAll (Example <S>) List <S>
findAll (Sort) List <T>
deleteInBatch (Iterable <T>) void
findAllById (Iterable <ID>) List <T>
deleteAllInBatch (Iterable <T>) void
deleteAllInBatch () void
flush() void
findAll (Example <S>, Sort) List <S>

Добавление зависимости Spring Data JPA

Зависимость Spring Data JPA добавляется в приложения Spring Boot с помощью начальной зависимости JPA. Она подключает саму библиотеку Spring Data JPA и, транзитивно, библиотеку Hibernate, реализующую механизм JPA:

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```

Выбор альтернативной реализации JPA:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-core</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.eclipse.persistence</groupId>
  <artifactId>org.eclipse.persistence.jpa</artifactId>
  <version>2.7.6</version>
</dependency>
```

Аннотирование классов данных предметной области

```
@Data
@Entity
public class Person{
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;

    @Size(min=1, message="You must choose at least 1 ingredient")
    @ManyToMany()
    private List<Address> addresses= new ArrayList<>();

    public void addAddress(Address address) {
        this.addresses.add(address);
    }
}
```

Объявление репозитория в JPA

```
public interface PersonRepository extends CrudRepository<Person, String> {  
}
```

или

```
public interface PersonRepository extends JpaRepository<Person, String> {  
}
```

Запросы в репозитории

Пример: нам необходимо получить список людей с заданным именем:

```
List<Person> findByName(String name);
```

Рассмотрим другой, более сложный пример:

```
List<Person> readPersonsByAddressZipAndStreetAndBuildingNumberBetween(  
String addressZip, String street, Integer startBuilding, Integer endBuilding);
```

Запросы в репозитории

```
List<Person>  
    read  
    Persons  
    By  
    AddressZip  
    AndStreet  
    AndBuildingNumberBetween(  
String addressZip, String street,  
Integer startBuilding, Integer  
endBuilding);
```

Этот метод должен прочитать (read) данные (также можно использовать «get» и «find»)

Отделяет свойства, по которым будет определяться соответствие предикату

Соответствие определять по свойству .addressZip или .address.zip а далее по свойству street

Значение свойства должно находиться между (between) заданными значениями ...и (and)...

Ключевые слова

В дополнение к неявной операции Equals (равно) и операции Between (между) сигнатуры методов репозитория Spring Data могут включать любой из следующих операторов:

- IsAfter, After, IsGreaterThan, GreaterThan;
- IsGreaterThnEqual, GreaterThanEqual;
- IsBefore, Before, IsLessThan, LessThan;
- IsLessThanEqual, LessThanEqual;
- IsBetween, Between;
- IsNull, Null;
- IsNotNull, NotNull;
- IsIn, In;
- IsNotIn, NotIn;
- IsStartingWith, StartingWith, StartsWith;
- IsEndingWith, EndingWith, EndsWith;
- IsContaining, Containing, Contains;
- IsLike, Like;
- IsNotLike, NotLike;
- IsTrue, True;
- IsFalse, False;
- Is, Equals;
- IsNot, Not;
- IgnoringCase, IgnoresCase;
- OrderBy

Использование аннотации @Query

```
@Query("Person p where p.addresses.city='Minsk'")  
List<Person> readPersonsFromMinsk();
```

Специализированные методы запросов также можно использовать совместно с Spring Data JDBC, но в этом случае необходимо учитывать следующие важные отличия:

- все специализированные методы запросов должны снабжаться аннотацией @Query. Это связано с тем, что, в отличие от JPA, Spring Data JDBC не поддерживает анализ имен методов и не может автоматически конструировать запросы;
- все запросы, указанные в @Query, должны быть SQL-запросами, а не запросами JPA.

Итоги

Spring Data JDBC и Spring Data JPA упрощают работу с реляционными базами данных, сводя ее к определению интерфейсов репозиториев.

Шаги по использованию Spring Data JPA:

1. Внедрение зависимости `spring-boot-starter-data-jpa`
2. Аннотирование класса сущности
3. Объявление репозитория (реализация интерфейсов Spring Data JPA)
4. Написание именованных запросов (если необходимо).

Источники

<https://proselyte.net/tutorials/jdbc/statements/>

<https://blog.skillfactory.ru/glossary/jdbc/>

<https://habr.com/ru/post/326614/>

<https://habr.com/ru/companies/otus/articles/531332/>

[Spring in Action \(6th edition\)](#)

[Документация Spring Data JPA](#)

[Baeldung - Spring Persistence » Spring Data JPA](#)